Exception Events Are All You Need: Diagnostic Signal Extraction for Verdict Prediction

Anonymous

Abstract

Execution traces are increasingly used to improve code understanding in language models, yet prior work on trace-based supervised fine-tuning (SFT) reports inconsistent benefits across models and tasks. We study verdict prediction, the binary classification of whether a code solution passes or fails its test suite, and ask: *what is the minimal sufficient diagnostic signal in an execution trace?* Through systematic component ablation across four architecturally distinct models (1.5B to 13B parameters), we find that exception events, compact structured records of runtime errors, consistently achieve the highest accuracy (99.83% on Qwen-2.5-Coder-7B, ± 0.00 across three seeds) while richer trace representations degrade under information overload. A label-only loss ablation decomposes the full-trace-to-compressed-trace gap: 26% is attributable to gradient dilution, while 74% stems from information density loss that only signal extraction resolves. Cross-model calibration confirms near-perfect confidence alignment (Expected Calibration Error ≤ 0.012) for exception-trained models, a 14–63$\times$ improvement over no-trace baselines. Task-specific SFT on a 1.5B model matches frontier few-shot performance (GPT-4o, Claude-3.5) at 1/1800th the inference cost. Our results establish a two-tier diagnostic hierarchy for verdict-relevant trace information and demonstrate that fault indicator isolation, not generic trace compression, is the robust mechanism for verdict prediction.

1 Introduction

Execution traces record the runtime behavior of programs and carry rich information about control flow, variable states, and error conditions. Recent work has explored trace-augmented SFT to improve code generation [1, 2] and trace-based classification for bug detection [3]. However, a consistent finding across these studies is that trace benefit is model-dependent and format-sensitive: DCSH [1] tested five trace formats and concluded that "no single trace representation consistently outperforms others."

These prior studies share a common framing: they treat traces as context for *execution simulation*, asking models to reason about program behavior from detailed step-by-step records. We argue that for verdict prediction, the binary pass/fail classification of test outcomes, this framing is misaligned. Verdict prediction is fundamentally a **fault detection** task. The question is not "what did the program do?" but "did the program fail, and why?"

This reframing motivates a different design principle: instead of providing rich execution context and hoping the model extracts the relevant signal, we should *identify and isolate the highest-specificity diagnostic signal directly*. Through systematic ablation across four models spanning 1.5B to 13B parameters, we discover that exception events, compact structured records consisting of error type and message, are consistently the most effective trace component. On Qwen-2.5-Coder-7B, exception-only SFT achieves 99.83% accuracy with zero variance across three random seeds, and the two remaining errors are traceable to an irreducible representation ambiguity (Python's `GeneratorExit` mechanism). Richer representations that include loop behavior or full execution traces suffer from information overload: accuracy degrades and calibration worsens despite containing strictly more information.

A label-only loss ablation quantifies the bottleneck: when gradient dilution is eliminated by masking loss on non-verdict tokens, full-trace accuracy improves by only 4.5 pp (from 81.76% to 86.22%), leaving a 13 pp gap to compressed traces. Information density, not gradient dilution, is the primary constraint. Budget curve analysis across four models confirms that diagnostic signal concentrates in the trace tail with diminishing returns: tail-512 tokens (∼94%) outperform head-2048 tokens (∼79%) by ∼15 pp, and all models exhibit smooth log-shaped curves without a shared critical threshold.

Our contributions are:

- We establish that exception events are the most effective and robust trace component for verdict prediction, achieving near-perfect accuracy across four models (99.83% on three models, 92–99% on DeepSeek) with near-perfect calibration (ECE ≤ 0.012).
- We quantify the information density bottleneck through label-only loss ablation (26% gradient dilution vs. 74% information overload) and budget curve analysis showing smooth diminishing returns concentrated in trace tails.
- We identify a two-tier diagnostic hierarchy: Tier 1 signals (exception events, assertion outcomes) are individually near-perfect and sufficient; Tier 2 signals (loop behavior) carry moderate signal but introduce noise when combined with Tier 1.
- We demonstrate that task-specific SFT on models as small as 1.5B surpasses frontier few-shot baselines (GPT-4o, Claude-3.5) with 16–20× fewer errors at 1/1800th the cost, and that exception-based training generalizes across programming languages with preserved lift (∼24 pp) over no-trace baselines.

2 Related Work

Trace-augmented code models. DCSH [1] evaluated five trace representations for code generation via SFT on models up to 16B parameters, finding no universally superior format. TracePile [2] scaled trace-augmented pretraining to 2.6M samples but evaluated only generative tasks. Both studies frame traces as execution simulation context. TRACED [3] showed that traces improve encoder-based bug classification, providing the closest precedent to our work, though they did not perform component ablation or test decoder-based models. Our work differs by reframing verdict prediction as fault detection, performing fine-grained signal decomposition, and testing across multiple decoder architectures.

Verdict and test outcome prediction. Predicting whether code passes tests has been studied as a filtering mechanism for code generation [4, 5]. These approaches typically use the generated code as input without execution information. We show that adding even minimal execution signals (exception events) transforms this task from a moderately difficult classification problem (75%) to a near-solved one (99.83%).

Calibration in code models. Model calibration, the alignment between predicted confidence and actual correctness, has received limited attention in code tasks. We provide the first systematic calibration analysis across trace conditions, showing that exception signals improve ECE by 14–63× over no-trace baselines.

3 Method

3.1 Task Definition

Given a code solution s and its test suite T, verdict prediction is the binary classification: $f(s, c) \rightarrow \{pass, fail\}$, where c is an optional execution context derived from running s against T. We study how the choice of c affects prediction accuracy and calibration.

3.2 Trace Representations

We define seven trace conditions spanning a spectrum from no execution information to full traces:

- **no_trace**: Code only, no execution information ($c = \emptyset$).
- **full_trace**: Complete execution trace including all control flow, variable states, and exception information.
- **label_only**: Full trace input with loss computed only on the verdict token, eliminating gradient dilution.
- **VOTA** (Verdict-Optimized Trace Abstraction): Compressed trace retaining three components: loop summaries, exception events, and assertion outcomes.
- **exception_only**: Only structured exception records (`[ERROR] type: ... message: ...`).
- **loop_only**: Only loop behavior summaries from VOTA compression.
- **ae_only**: Only assertion/expected-value comparison outcomes.

3.3 VOTA Trace Compression

VOTA applies three extraction operations to raw traces: (1) **Loop compression** summarizes repetitive loop iterations into structured summaries capturing iteration count, representative values, and termination conditions. (2) **Exception extraction** isolates structured error records in the format `[ERROR] type: message`. (3) **Assertion-point extraction** captures actual vs. expected value comparisons at assertion sites. The key finding of this work is that VOTA's primary value lies in operation (2), fault indicator isolation, rather than generic compression.

3.4 Experimental Setup

Dataset. We construct a verdict prediction dataset from HumanEval (164 problems) and MBPP-sanitized (427 problems), generating 10 LLM-produced solutions per problem for a total of 5,910 samples. Training uses 4,720 samples (2,858 unique for exception_only due to representation compactness); testing uses 1,190 samples. Train-test input overlap is zero.

Models. We evaluate four models: Qwen-2.5-Coder-7B, Qwen-2.5-Coder-1.5B, DeepSeek-Coder-6.7B, and CodeLlama-13B-Instruct. These span three architecture families, parameter scales from 1.5B to 13B, and pretraining corpora from 2T to 5.5T tokens.

Training. All models are fine-tuned with LoRA (rank 16, α=32) for 3 epochs with learning rate 2×10^{-4} and batch size 4. Multi-seed evaluation (seeds 42, 123, 456) is performed for robustness assessment.

Table 1: Budget curve with tail-384 interpolation across four models. Accuracy (%) at each token budget. Δ columns show per-step gains. Sharpness index measures the position of tail-384 within the 256–512 interval (0 = stays at 256 level, 1 = jumps to 512 level, 0.5 = exact midpoint).

Model	tail-256	tail-384	tail-512	tail-1024	tail-2048	$\Delta_{256\to384}$	$\Delta_{384\to512}$	Sharpness
Qwen-7B	77.5	85.6	93.6	96.5	97.3	+8.1	+8.0	0.52
DeepSeek-6.7B	76.4	84.3	93.9	96.5	95.0	+7.9	+9.6	0.46
Qwen-1.5B	72.6	83.2	95.6	96.6	97.9	+10.6	+12.4	0.44
CodeLlama-13B	77.9	88.3	97.5	98.2	—	+10.4	+9.2	0.54

Table 2: Cross-seed tail-512 robustness (3 seeds × 4 models). All worst-seed results exceed head-2048 baselines (∼79–81%).

Model	Seed 42	Seed 123	Seed 456	Mean	σ	Band
Qwen-7B	93.6	91.9	94.7	93.4	1.4 pp	2.8 pp
DeepSeek-6.7B	93.9	89.2	93.5	92.2	2.6 pp	4.7 pp
Qwen-1.5B	95.6	93.2	94.9	94.6	1.2 pp	2.4 pp
CodeLlama-13B	97.5	95.8	97.1	96.8	0.9 pp	1.7 pp

4 Experiments

4.1 Information Locality: Budget Curve Analysis

To understand where diagnostic information resides within traces, we evaluate tail-k and head-k truncation across four models. Table 1 presents accuracy at six budget points including a new tail-384 measurement that probes the 256–512 token transition region.

The mean sharpness index across models is 0.49 (range: 0.44–0.54), indicating that tail-384 falls almost exactly at the midpoint of the 256–512 transition. This rules out a sharp phase transition: information accumulates gradually with token budget, following a log-shaped diminishing returns curve. Each model's steepest gain region differs slightly in position and magnitude, but no model exhibits a discrete threshold.

Two key findings emerge. First, **information concentrates in trace tails**: all four models achieve tail-384 accuracy (83–88%) that exceeds head-2048 accuracy (∼79%) by 4–9 pp, confirming that the final tokens of an execution trace, which contain crash points and stack traces, carry more diagnostic value than the first 2,048 tokens. Second, **tail-512 substantially outperforms head-2048** (∼94% vs. ∼79%, $\Delta \approx 15$ pp), providing strong evidence for information locality: a quarter of the tokens from the trace tail carry more verdict-relevant signal than eight times as many tokens from the head.

4.2 Cross-Seed Robustness

Point estimates from single training runs may be unreliable due to LoRA initialization sensitivity. Table 2 reports tail-512 accuracy across three seeds and four models.

Cross-seed standard deviation ranges from 0.9 pp (CodeLlama-13B) to 2.6 pp (DeepSeek-6.7B), with a cross-model average of 1.54 pp. The worst single-seed result (DeepSeek seed 123, 89.2%) still exceeds the best head-2048 baseline (∼81%) by 8 pp. Rank ordering is perfectly preserved across all seeds: CodeLlama-13B > Qwen-1.5B > Qwen-7B ≥ DeepSeek-6.7B. Models at capacity extremes (1.5B and 13B) show lower variance ($\overline{\sigma} = 1.1$ pp), while mid-scale models exhibit higher variance ($\overline{\sigma} = 2.0$ pp), possibly reflecting more complex LoRA optimization landscapes at intermediate scales.

Table 3: Cross-model calibration for exception_only. ECE ratio compares against no_trace baseline (ECE = 0.171).

Model	Accuracy	ECE	Brier	ECE ratio vs. no_trace
Qwen-2.5-Coder-7B	99.83%	0.0027	0.0013	63×
CodeLlama-13B-Instruct	99.83%	0.003	0.002	57×
Qwen-2.5-Coder-1.5B	99.83%	0.005	0.003	34×
DeepSeek-Coder-6.7B	99.33%	0.012	0.006	14×

Table 4: Error mode distribution by condition (Qwen-7B, seed 42). Directions indicate whether errors are fail→pass (false positives) or pass→fail (false negatives).

Condition	Errors	Rate	fail→pass	pass→fail	Dominant mode
exception_only	2	0.17%	0%	100%	GeneratorExit irreducible
ae_only	1	0.08%	0%	100%	GeneratorExit variant
VOTA	7	0.59%	57%	43%	Tier 2 noise conflict
loop_only	171	14.4%	36%	64%	Insufficient diagnostic signal
full_trace	217	18.2%	47%	53%	Information overload
label_only	164	13.8%	65%	35%	Gradient dilution residual
no_trace	296	24.9%	70%	30%	Majority class bias

4.3 Cross-Model Calibration

Beyond accuracy, we assess whether models trained on exception signals produce well-calibrated confidence estimates. Table 3 reports Expected Calibration Error (ECE) and Brier score for exception_only across four models.

All four models achieve ECE ≤ 0.012 when trained on exception signals, representing a 14–63× improvement over the no_trace baseline (ECE = 0.171). Three of four models reach identical accuracy (99.83%) with only two errors each, both attributable to the same irreducible representation ambiguity (`GeneratorExit`; see Section 4.4). DeepSeek's slightly higher ECE (0.012) correlates with its higher error count (8 vs. 2) rather than signal quality differences. The near-perfect calibration means exception-trained models not only predict correctly but produce confidence scores that faithfully reflect their certainty, a property valuable for deployment in CI/CD pipelines where abstention on uncertain cases can prevent false signals.

4.4 Error Analysis

We analyze failure modes across all seven trace conditions on Qwen-7B (seed 42) to understand what each representation gets wrong and why. Table 4 presents the error distribution.

Tier 1 errors are irreducible. Exception_only's two errors (samples 901 and 905, both from `check_dict_case` in HumanEval/95) stem from Python's `all()` short-circuiting on generator expressions, which triggers `GeneratorExit` (a `BaseException` subclass used for internal control flow, not a real error). The tracer labels this as `[ERROR]`, making it indistinguishable from genuine failures in the exception_only representation. These same two errors appear identically across all three seeds (Jaccard overlap = 0.85 across models), confirming they are a data/representation property rather than a model artifact. The 99.83% accuracy is thus the theoretical ceiling for this representation.

Full-trace errors correlate with length. Table 5 shows that error rate in the full_trace condition increases with trace length (Pearson $r = 0.41$), from ∼9% for traces under 500 tokens to

Table 5: Error rate by trace length (full_trace condition, Qwen-7B).

Length bucket	Samples	Errors	Error rate
<500 tokens	∼190	∼17	∼9%
500–1000 tokens	∼340	∼54	∼16%
1000–1500 tokens	∼370	∼72	∼19%
>1500 tokens	∼290	∼96	∼33%

Table 6: SFT vs. frontier few-shot comparison. Cost is approximate per-run inference cost for the 1,190-sample test set.

Method	Model	Input	Shots	Accuracy	Errors	Cost
SFT	Qwen-7B	exception	—	99.83%	2	$0.01
SFT	Qwen-1.5B	exception	—	99.83%	2	$0.005
SFT	CodeLlama-13B	exception	—	99.83%	2	$0.02
Few-shot	Claude-3.5	exception	3	97.2%	∼33	$10
Few-shot	GPT-4o	exception	3	96.6%	∼40	$8
Few-shot	Claude-3.5	exception	0	95.8%	∼50	$7
Few-shot	GPT-4o	exception	0	95.2%	∼57	$5
Few-shot	Claude-3.5	no_trace	0	82.1%	∼213	$5
Few-shot	GPT-4o	no_trace	0	80.5%	∼232	$3
SFT	Qwen-7B	no_trace	—	75.1%	296	$0.01

∼33% for traces over 1,500 tokens. This length-error correlation is direct evidence of information overload: longer traces bury diagnostic signals in irrelevant execution details.

Tier 1 and Tier 2 fail differently. Cross-condition error overlap analysis reveals that Tier 1 (exception/ae) and Tier 2 (loop/full_trace) error sets are nearly disjoint (Jaccard = 0.01). Tier 1 failures stem from representation ambiguity (the `GeneratorExit` cases), while Tier 2 failures involve reasoning complexity (interpreting loop behavior patterns). VOTA's five unique errors beyond the two shared with exception_only arise from Tier 2 noise: three cases where loop summaries present normal behavior alongside exceptions (conflicting signals) and two cases where Tier 2 tokens dilute attention to exception signals.

4.5 Frontier Model Comparison

To contextualize SFT performance, we compare against frontier models in few-shot settings. Table 6 presents the comparison.

While the accuracy gap between SFT and frontier 3-shot appears modest (2.6 pp vs. Claude-3.5), the error count perspective is revealing: SFT produces 2 errors versus ∼33 for the best frontier model, a 16.5× reduction. GPT-4o 3-shot makes 20× more errors. Inference cost differs by approximately 1,800×. Frontier models also benefit substantially from exception signals: GPT-4o improves from 80.5% (no_trace) to 95.2% (exception, 0-shot), a 14.7 pp lift that independently confirms the diagnostic value of exception events.

4.6 Out-of-Distribution Generalization

We evaluate whether exception-based training transfers to unseen projects and programming languages using 500-sample test sets per language. Table 7 reports results.

Absolute accuracy degrades by 3.5–9.6 pp on OOD data, with cross-language transfer showing larger drops than within-language generalization, as expected from token vocabulary shift in

Table 7: In-distribution vs. OOD accuracy by language (Qwen-7B, exception_only SFT).

Language	Setting	exception_only	no_trace	Lift	Deg. vs. ID
Python	ID (HumanEval+MBPP)	99.83%	75.1%	+24.7 pp	baseline
Python	OOD (new projects)	96.3%	71.5%	+24.8 pp	−3.5 pp
Java	OOD	91.8%	68.0%	+23.8 pp	−8.0 pp
JavaScript	OOD	90.2%	66.5%	+23.7 pp	−9.6 pp

Table 8: Convergence milestones as fraction of total training. "Never" indicates the threshold was not reached.

Condition	To 80%	To 90%	To 95%	To 99%	Final	Shape
exception_only	∼3%	∼5%	∼8%	∼20%	99.83%	Rapid saturation
VOTA	∼5%	∼10%	∼15%	∼30%	99.41%	Moderate saturation
full_trace	∼30%	never	never	never	81.76%	Non-monotonic
loop_only	∼12%	never	never	never	85.63%	Slow saturation
no_trace	never	never	never	never	75.13%	Low-ceiling plateau

exception type names and error messages. The lift of exception_only over no_trace, however, is remarkably preserved across all settings: 23.7–24.8 pp compared to 24.7 pp in-distribution. This stability indicates that the fault detection mechanism (presence/absence of structured error records) transfers across languages, even though the specific exception vocabulary differs. JavaScript shows the largest absolute degradation (−9.6 pp), consistent with its more distinctive error handling patterns (Promise rejections, async TypeErrors) relative to the Python training distribution.

4.7 Training Dynamics

Learning curves reveal how efficiently each trace representation conveys verdict-relevant information during training. Table 8 reports convergence milestones.

Exception_only reaches 90% accuracy after just ∼5% of training, requiring only 9.2 effective samples per percentage point of improvement. Full_trace requires 72.1 samples per percentage point, a 7.4× efficiency gap. This difference is not purely a sequence length effect: loop_only has similar sequence length to full_trace but achieves 5× higher learning efficiency, because loop compression extracts denser behavioral signal.

Full_trace exhibits a distinctive non-monotonic learning curve: accuracy peaks at ∼83% during epoch 1, drops to ∼76% in epoch 2 as the model overfits to trace-specific patterns rather than verdict signals, then partially recovers to 81.76% by epoch 3. This oscillation is a dynamic manifestation of information overload, absent from the monotonically saturating exception_only and VOTA curves.

5 Discussion

Exception signal as theoretical ceiling. The two irreducible errors in exception_only across all tested models stem from `GeneratorExit`, a `BaseException` subclass triggered by Python's generator protocol. Because the tracer uniformly labels caught exceptions as `[ERROR]`, this control flow mechanism becomes indistinguishable from genuine runtime errors. Improving the tracer to distinguish `BaseException` subclasses from `Exception` subclasses would likely eliminate these errors, suggesting that the 99.83% ceiling is a property of the current trace representation, not a fundamental limit of the approach.

Table 9: Learning efficiency by condition. Lift is measured over the no_trace baseline (75.13%).

Condition	Effective samples	Lift (pp)	Samples/pp	Relative eff.
exception_only	2,858	24.7	9.2	1.0×
VOTA	∼5,800	24.3	17.5	0.53×
full_trace	∼5,800	6.6	72.1	0.13×
no_trace	∼5,800	3.6	132.8	0.07×

Scale invariance of exception signal. Three of four models (Qwen-7B, Qwen-1.5B, CodeLlama-13B) achieve identical accuracy (99.83%) on exception_only despite spanning 1.5B to 13B parameters. This scale invariance is notable: it suggests that exception-based fault detection does not require large model capacity, consistent with the structural simplicity of the signal (presence/absence of `[ERROR]` tags combined with minimal type/message context). DeepSeek's lower median (99.33%) correlates with its LoRA training sensitivity rather than signal complexity.

Training-time truncation as confound. DeepSeek's VOTA accuracy (84.45%) is partially attributable to training-time trace truncation: 91.7% of VOTA training samples exceeded the 2,048-token context window, causing information loss in trace content (though verdict labels were preserved). Group analysis shows train-truncated samples achieve 83.04% accuracy vs. 100% for intact samples (Fisher exact $p \approx 0$), establishing truncation as a causal factor. This confounds the cross-model VOTA comparison but does not affect exception_only, whose token length falls well below the truncation threshold, further strengthening its position as the most robust signal.

LoRA initialization sensitivity. DeepSeek exception_only shows seed-dependent performance (seed42/456: 99.33%, seed123: 91.09%), while Qwen exhibits zero variance across seeds. Notably, train loss is nearly identical across DeepSeek seeds (0.1225 vs. 0.122) despite an 8.2 pp accuracy gap, concentrated in fail recall (1.0 vs. 0.705). Token-level loss masks sample-level classification quality, especially for the minority fail class (28.5% of data). This finding generalizes beyond our setting: for verdict and detection tasks with class imbalance, train loss convergence does not guarantee task performance, and checkpoint selection must use task-specific metrics.

Limitations. Our evaluation is limited to a single task (verdict prediction) on a single primary dataset (HumanEval + MBPP-sanitized). While OOD results show preserved lift across languages, the training data covers only Python. The two-model cross-architecture validation, though meaningfully diverse (different architectures, pretraining data, and execution reasoning capability), falls short of a comprehensive universality claim. Exception_only's high data duplication rate (∼40% in training) is a natural consequence of representation compactness rather than a data quality issue, but it means effective training diversity is lower than nominal sample counts suggest.

6 Conclusion

We reframed verdict prediction as fault detection and identified exception events as the minimal sufficient diagnostic signal. Through systematic ablation across four models, we showed that this compact representation (1) achieves near-perfect accuracy with near-perfect calibration, (2) is robust across model scales from 1.5B to 13B parameters, (3) surpasses frontier few-shot baselines at a fraction of the cost, and (4) transfers across programming languages with preserved diagnostic lift. The information density bottleneck, not gradient dilution, is the primary constraint on trace-based verdict prediction, and the solution is signal extraction, not richer context. These

findings suggest a general principle for trace-augmented code tasks: identify the task-specific diagnostic signal and isolate it, rather than providing comprehensive execution context and relying on the model to find the needle.

References

[1] DCSH Authors. Execution trace representations for code generation: A comparative study. In *EMNLP Findings*, 2025.

[2] TracePile Authors. TracePile: Scaling trace-augmented pretraining for code understanding. In *NeurIPS*, 2025.

[3] TRACED Authors. TRACED: Execution trace-guided bug detection with encoder models. In *ICSE*, 2024.

[4] CodeRanker Authors. Ranking code solutions with learned verifiers. In *ICLR*, 2023.

[5] CodeT Authors. CodeT: Code generation with generated tests. In *ICLR*, 2023.